

Serial Digital Communication

The Core Building Blocks
of Embedded Connectivity

The Hardware Constraint

Parallel Communication

- Multiple dedicated wires
- Sends multiple bits simultaneously
- Extremely fast



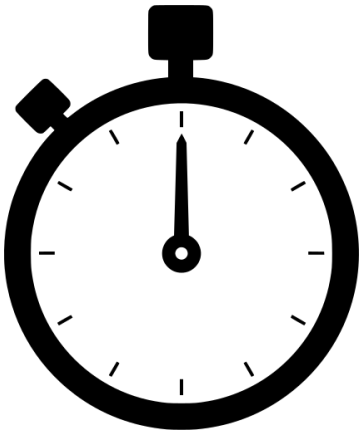
Serial Communication

- Vastly reduced physical footprint.
- Sends bits sequentially over a single data line
- Universal standard for chip-to-chip comms.

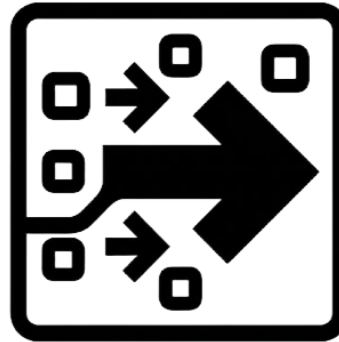


The Three Core Variables

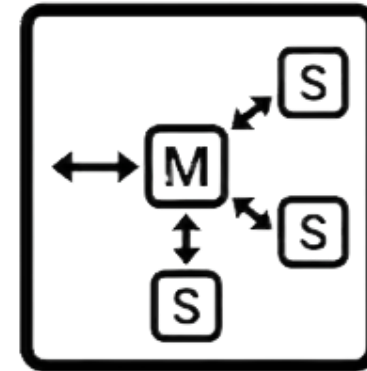
Every specific serial protocol is simply a unique combination of three fundamental engineering choices.



TIMING



DATA FLOW

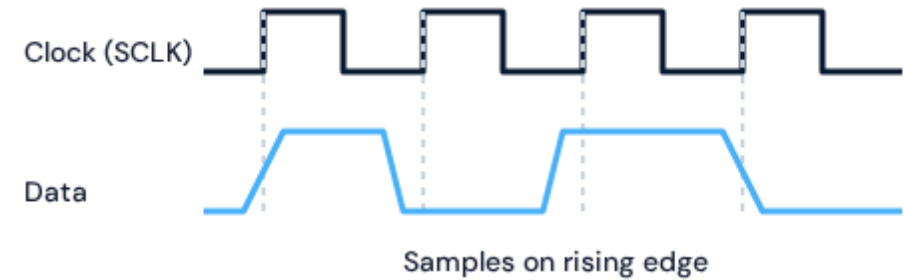


TOPOLOGY

Variable 1: Timing

Synchronous

Devices share a dedicated physical **Clock Wire (SCLK)**. The receiver exactly knows when to read the data line because it watches for the clock line to transition.



Asynchronous

No shared clock line. Both devices must be pre-configured to the exact same speed (Baud rate). Uses "Start" and "Stop" bits wrapped around the payload.

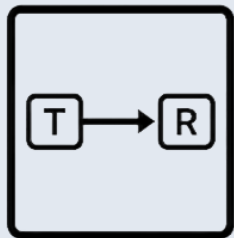


Variable 2: Direction of Data Flow



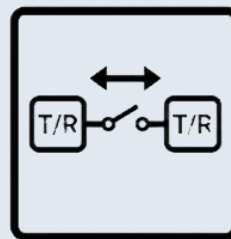
Simplex

A strict one-way street. Data only ever flows from the transmitter to the receiver.



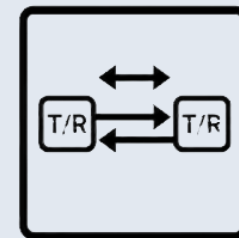
Half-Duplex

A two-way street, but only one device can talk at a time over a shared line.

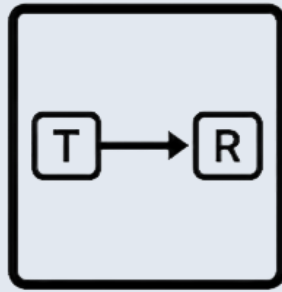


Full-Duplex

Simultaneous two-way communication. Requires dedicated TX and RX lines.

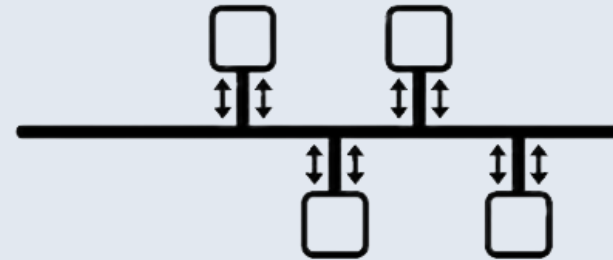


Variable 3: Topology



Point-to-Point

A direct, dedicated 1-to-1 connection. Simple and secure, but doesn't scale well if you need to connect many devices to a single microcontroller.



Bus (Multi-drop)

One Controller communicates with multiple Peripherals on shared wires. Requires hardware selection pins or software addressing to route messages correctly.

The "Big Three" Protocols

Applying our framework to UART, SPI, and I2C to deeply understand how they work.

UART: Configuration & Purpose

The Configuration

- **Timing:** Asynchronous
- **Direction:** Full-Duplex (Usually)
- **Topology:** Point-to-Point
- **Typical Speed:** 9.6 kbps – 115.2 kbps

UART (Universal Asynchronous Receiver–Transmitter) is not technically a protocol, but a physical circuit in a microcontroller that translates data between parallel and serial forms.

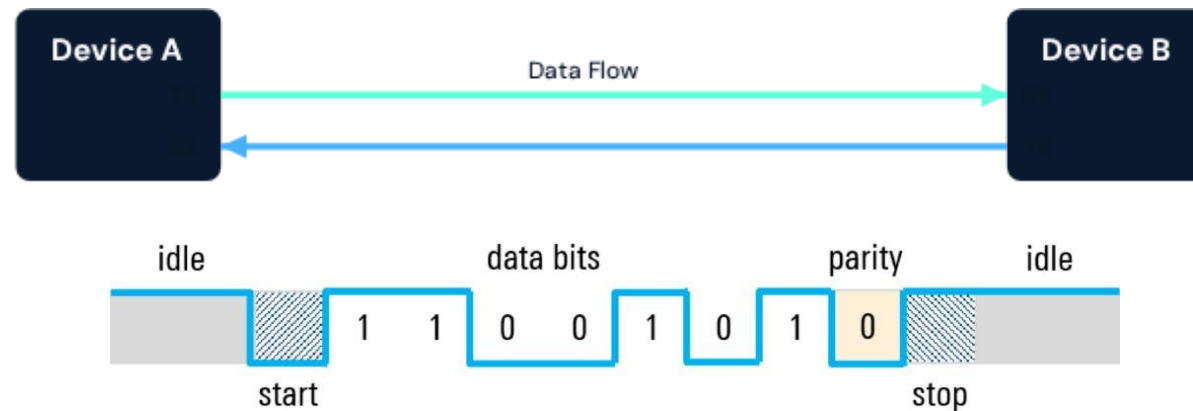
Why choose UART?

Minimalism. Two wires (TX and RX) are the bare minimum needed for two devices to talk back and forth simultaneously. By dropping the clock wire, you save a physical pin on the microcontroller.

Real-World Uses: Because it operates at slower, reliable speeds without a clock, it's perfect for Legacy sensors (GPS, RFID), Bluetooth modules, and simple debug consoles.

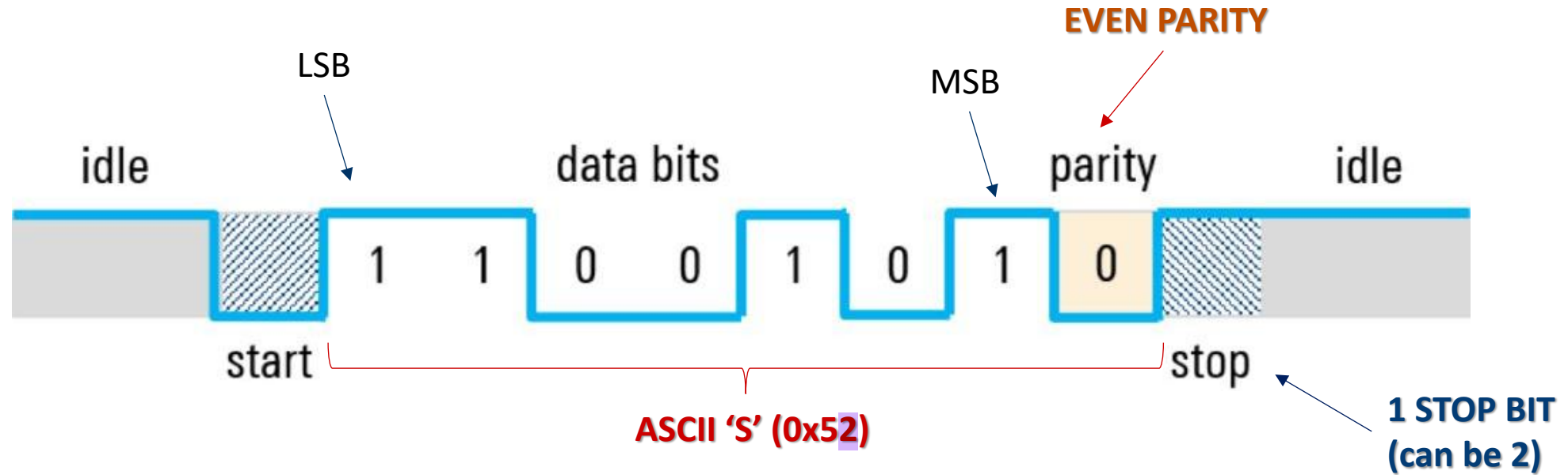
UART: How it Works

Because there is no clock wire to keep them synchronized, both the sender and receiver must be pre-configured to talk at the exact same speed (the **Baud Rate**, e.g., 9600 or 115200 bits per second). Devices are cross-wired: TX connects to RX.



Line is held HIGH when idle. A drop to LOW signals the start of transmission

Baud Rate Calculation for UART



It uses **7** data bits + **3** bits for control = **10** bits per character.
 So, if the transmission rate is **100** characters per second, the baud rate will be:

$$100 \times 10 = 1000 \text{ baud/s, or } 1000 \text{ bps}$$

UART: Variations & Considerations

1. Flow Control

What happens if Device A sends data faster than Device B can process it? Buffers overflow.

Hardware Flow Control: Adds two extra wires (RTS – Request to Send, and CTS – Clear to Send) so devices can pause each other.

Software Flow Control: Sends special characters (XON/XOFF) inside the data stream to pause/resume transmission without extra wires.

2. Distance vs. Noise (RS-232 / RS-485)

Standard UART uses TTL logic levels (0V and 3.3V/5V), which are highly susceptible to electrical noise over long cables.

RS-232: Uses higher voltage swings (e.g., -15V to +15V) to increase range.

RS-485: Converts the UART signal into *differential signaling* over twisted pairs, allowing communication over thousands of feet in noisy industrial environments.

RP2040 UART Features

2x UART Peripherals

- UART0
- UART1

Data Length:

- 5 to 8 data bits

Parity:

- None
- Even
- Odd

Stop Bits:

- 1 bit
- 2 bits

TX and RX FIFOs

- 32-byte transmit FIFO
- 32-byte receive FIFO

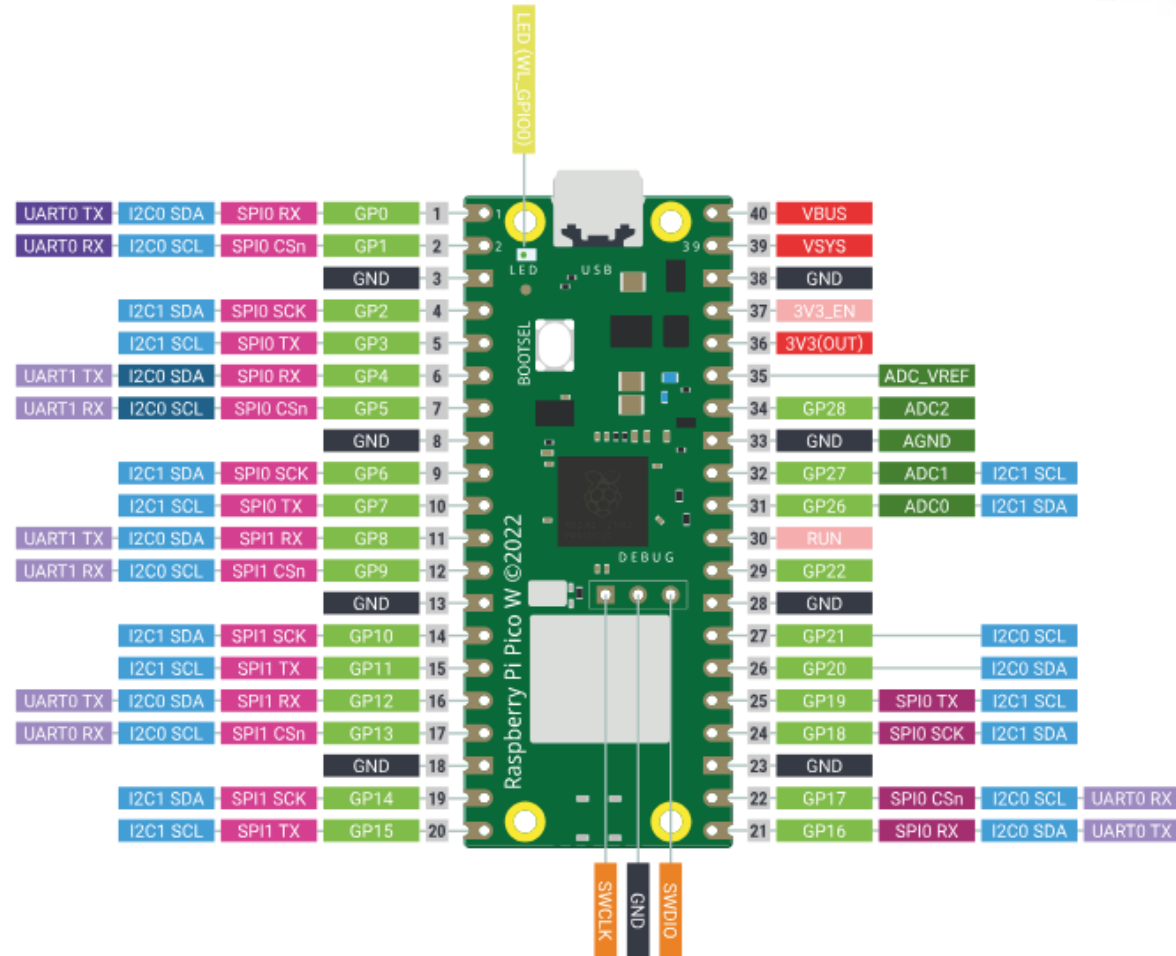
Interrupt Support

- TX FIFO level
- RX FIFO level
- Receive timeout
- Error conditions

DMA Support

Flexible Baud-rate Divider

Flow Control (CTS, RTS)



SPI: Configuration & Purpose

The Configuration

- **Timing:** Synchronous
- **Direction:** Full-Duplex
- **Topology:** Bus (Single-Master, Multi-Slave)
- **Typical Speed:** 10 MHz – 50+ MHz

SPI (Serial Peripheral Interface) was developed by Motorola in the 1980s as a high-speed backbone for embedded systems.

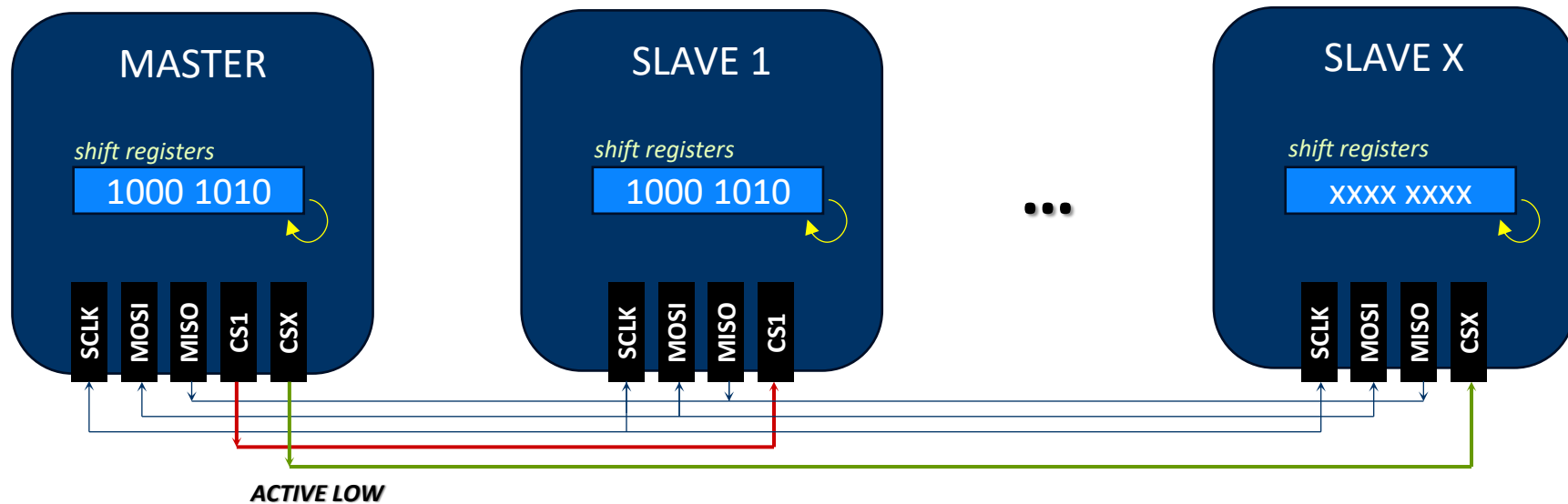
Why choose SPI?

Raw Speed. Because it has a dedicated clock line and push-pull drivers, massive amounts of data can flow continuously. The separate transmit and receive lines mean data flows both ways simultaneously.

Real-World Uses: Applications requiring enormous data pipelines like high-resolution LCDs, OLED displays, SD cards, and fast ADCs (Analog-to-Digital Converters).

SPI: How it Works

SPI acts like a circular shift register. On every clock pulse generated by the Master, one bit is pushed out on MOSI (Master Out Slave In) while simultaneously one bit is pulled in on MISO (Master In Slave Out). The Master uses a dedicated Chip Select (CS) pin to "wake up" the specific slave it wants to talk to.



SPI Modes

Clock Polarity and Phase (CPOL/CPHA)

Because there is no universal standard for SPI, manufacturers design chips differently. You must configure the Master to match the Slave's expected behavior.

- **CPOL (Polarity):** Is the clock line **idling** HIGH (1) or LOW (0) when **nothing is being sent**?
- **CPHA (Phase):** Is data captured on the *first* edge (transition) of the clock, or the *second* edge?

This creates 4 distinct "SPI Modes" (Mode 0, 1, 2, 3).

Mode	CPOL	CPHA	Idle Clock	Sample Edge
0	0	0	Low	Rising
1	0	1	Low	Falling
2	1	0	High	Falling
3	1	1	High	Rising



RP2040 SPI Features

2× SPI Peripherals

- SPI0
- SPI1

Operation Modes

- Master Mode
- Slave Mode

Data Format

- 4 to 16 data bits per frame
- MSB-first or LSB-first
- SPI Modes 0, 1, 2, and 3
 - CPOL = 0 or 1
 - CPHA = 0 or 1

SPI Signals

- SCK (Serial Clock)
- TX / MOSI (Master Out Slave In)
- RX / MISO (Master In Slave Out)
- CSn / SS (Chip Select)

TX and RX FIFOs

- 8-entry transmit FIFO
- 8-entry receive FIFO

Interrupt Support

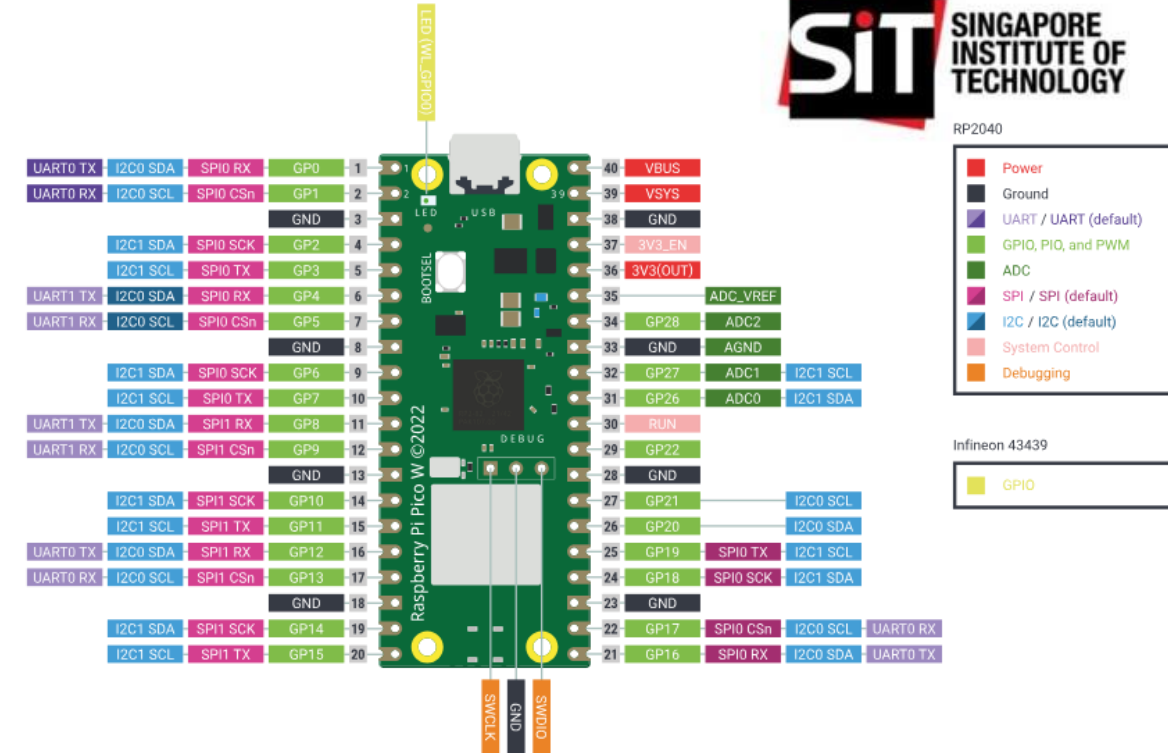
- TX FIFO level
- RX FIFO level
- RX timeout
- Receive overrun
- Receive underflow
- Transmit overflow

DMA Support

- DMA request for TX FIFO
- DMA request for RX FIFO

Flexible Clock Generation

- Programmable SPI clock divider
- Up to system clock limitations



Multi-Master Support

- Supported by hardware
- External arbitration required by system design

Full-Duplex Communication

- Simultaneous transmit and receive
- One bit shifted out and one bit shifted in per clock cycle

I²C: Configuration & Purpose

The Configuration

- **Timing:** Synchronous
- **Direction:** Half-Duplex
- **Topology:** Bus (Multi-Master, Multi-Slave)
- **Typical Speed:** 100 kHz (Standard) – 400 kHz (Fast)

I²C (Inter-Integrated Circuit, pronounced "I-squared-C") was invented by Philips Semiconductor to allow multiple chips on a TV board to communicate easily.

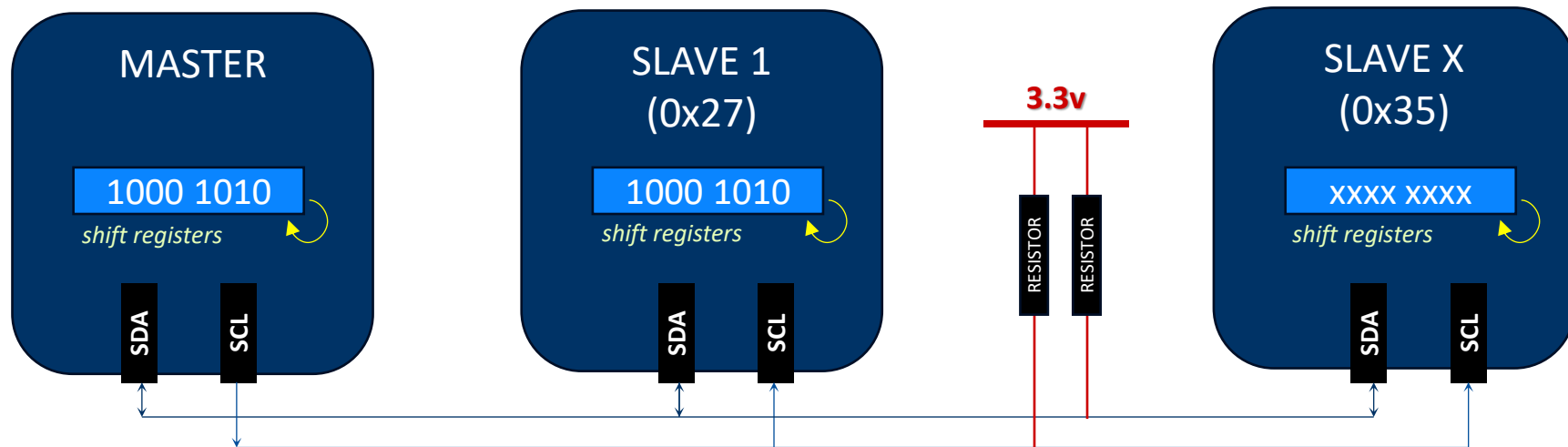
Why choose I²C?

Pin Conservation. You can connect over 100 devices to a microcontroller using exactly two wires (SCL and SDA). No dedicated Chip Select pins are required.

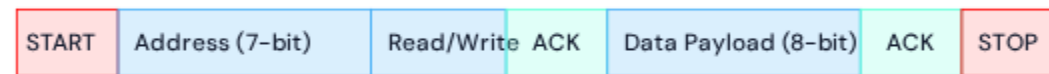
Real-World Uses: Because physical pull-up resistors limit the switching speed, it's perfect for low-bandwidth sensors that don't need to send much data (temperature, humidity, accelerometers), and Real Time Clocks (RTCs).

I²C: How it Works

I²C uses "open-drain" lines pulled HIGH to power by resistors. Devices can **only pull the lines LOW**. To talk to a specific slave, the Master broadcasts a **7-bit software address over the data line**. Only the **slave** with that matching hardcoded address responds (ACK), and data transfer begins.



I²C Data Packet Structure on SDA line:



1. Clock Stretching

Usually, the Master controls the clock (SCL) entirely. However, if a Slave is too slow to process the data (e.g., an ADC taking a reading), the Slave can physically hold the SCL line LOW.

The Master will see the clock line is low and wait. Once the Slave is ready, it releases the SCL line, it floats back HIGH via the pull-up resistor, and the Master resumes generating the clock. This allows slow devices to pause fast masters.

2. Multi-Master Arbitration

What if two Masters try to talk on the bus at the exact same time?

Because the lines are open-drain, pulling a line LOW always wins over letting it float HIGH. Both masters monitor the SDA line as they transmit. If Master A tries to send a '1' (letting it float HIGH), but sees the line go LOW, it knows Master B is transmitting a '0'. Master A instantly backs off and lets Master B finish.

RP2040 I²C Features

2× I²C Peripherals

- I²C0
- I²C1

Operation Modes

- Controller (Master) Mode
- Target (Slave) Mode
- Multi-Master Capable

Addressing

- 7-bit Addressing
- 10-bit Addressing

Data Format

- 8-bit Data Bytes
- ACK/NACK Handshaking
- START / REPEATED START / STOP Conditions

I²C Signals

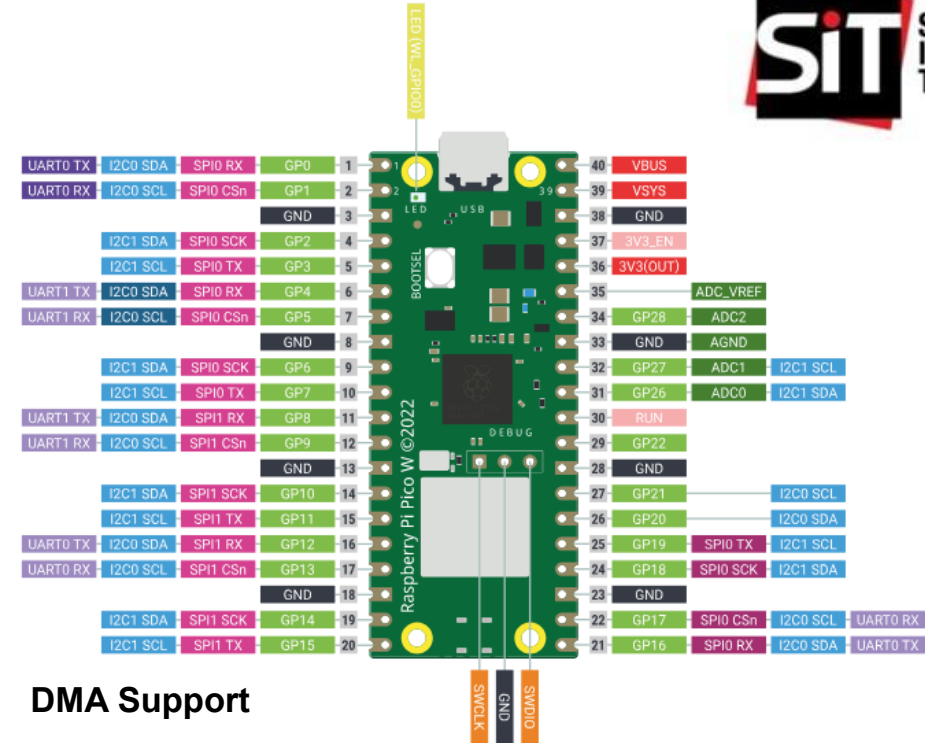
- SCL (Serial Clock)
- SDA (Serial Data)

TX and RX FIFOs

- 16-entry transmit FIFO
- 16-entry receive FIFO

Interrupt Support

- TX FIFO level
- RX FIFO level
- START detection
- STOP detection
- Read request detection
- RX overrun
- RX underrun
- TX overrun
- Abort conditions



DMA Support

- DMA request for TX FIFO
- DMA request for RX FIFO

Bus Features

- Standard Mode (100 kbps)
- Fast Mode (400 kbps)
- Fast Mode Plus (1 Mbps)
- Clock Stretching
- Arbitration Support

RP2040



Infineon 43439

GPIO

Protocol Selection: Speed vs. Application



UART (Slow)

~9.6 kbps to 115.2 kbps

Ideal for: Legacy sensors (GPS, RFID) and simple debug consoles.

Why? The lack of a shared clock makes synchronization harder at high speeds. It is chosen when wiring simplicity is more important than massive data throughput.



I²C (Medium)

100 kHz to 400 kHz

Ideal for: Environmental sensors (temperature, humidity) and RTCs.

Why? Because it relies on pull-up resistors to bring the voltage back HIGH, bus capacitance physically limits how fast the signal can cleanly switch.



SPI (Fast)

10 MHz to 50+ MHz

Ideal for: LCD screens, OLEDs, SD Cards, and high-speed ADCs.

Why? Push-pull hardware drivers forcefully drive the lines HIGH and LOW. Combined with a dedicated clock wire, this allows for massive, continuous data pipelines.

RP2040 Digital Communications

Feature	UART	SPI	I ² C
RP2040 Peripherals	2	2	2
Signals	TX, RX	SCK, MOSI, MISO, CS	SCL, SDA
Clock Required	No	Yes	Yes
Duplex	Full	Full	Half
Addressing	No	No	Yes (7/10-bit)
Multiple Devices	Difficult	Via CS lines	Native bus addressing
Data Size	5-8 bits	4-16 bits	8-bit bytes
Parity	Yes	No	No
Stop Bits	Yes	No	No
ACK/NACK	No	No	Yes
FIFO	32-byte	8-entry	16-entry
DMA	Yes	Yes	Yes
Multi-Master	No	Possible but uncommon	Supported
Typical Speed	kbps-Mbps	Mbps-tens of Mbps	100 kbps-1 Mbps